

Лабораторная работа №8

Оптимизация

Александр Эдуардович Аскеров

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Повтор примеров	8
3.2	Задания для самостоятельного выполнения	26
4	Выводы	34
	Список литературы	35

Список иллюстраций

3.1	Линейное программирование 1	8
3.2	Линейное программирование 2	9
3.3	Линейное программирование 3	9
3.4	Векторизованные ограничения и целевая функция оптимизации 1 .	10
3.5	Векторизованные ограничения и целевая функция оптимизации 2 .	10
3.6	Векторизованные ограничения и целевая функция оптимизации 3 .	11
3.7	Оптимизация рациона питания 1	12
3.8	Оптимизация рациона питания 2	12
3.9	Оптимизация рациона питания 3	13
3.10	Оптимизация рациона питания 4	14
3.11	Оптимизация рациона питания 5	15
3.12	Оптимизация рациона питания 6	15
3.13	Путешествие по миру 1	16
3.14	Путешествие по миру 2	17
3.15	Путешествие по миру 3	17
3.16	Портфельные инвестиции 1	18
3.17	Портфельные инвестиции 2	18
3.18	Портфельные инвестиции 3	19
3.19	Портфельные инвестиции 4	20
3.20	Портфельные инвестиции 5	21
3.21	Портфельные инвестиции 6	22
3.22	Портфельные инвестиции 7	22
3.23	Восстановление изображения 1	23
3.24	Восстановление изображения 2	24
3.25	Восстановление изображения 3	25
3.26	Восстановление изображения 4	26
3.27	Первый номер 1	27
3.28	Первый номер 2	27
3.29	Второй номер 1	28
3.30	Второй номер 2	28
3.31	Третий номер 1	29
3.32	Третий номер 2	30
3.33	Третий номер 3	30
3.34	Четвёртый номер 1	31
3.35	Четвёртый номер 2	32
3.36	Пятый номер 1	33

3.37	Пятый номер 2	33
------	-------------------------	----

Список таблиц

1 Цель работы

Основная цель работы – освоить пакеты Julia для решения задач оптимизации.

2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 8.2.
2. Выполните задания для самостоятельной работы.

3 Выполнение лабораторной работы

3.1 Повтор примеров

Повторим примеры из раздела линейное программирование. [1]

```
1 # Подключение пакетов
2 import Pkg
3 Pkg.add("JuMP")
4 Pkg.add("GLPK")
5 using JuMP
6 using GLPK

Updating registry at `~/.julia/registries/General.toml`
Resolving package versions...
Installed CodecBzip2 — v0.8.5
Installed StructTypes — v1.11.0
Installed JSON3 — v1.14.3
Installed MathOptInterface — v1.48.0
Installed JuMP — v1.29.3
```

Рисунок 3.1: Линейное программирование 1


```

1 # Определение объекта модели с именем model
2 model = Model(GLPK.Optimizer)

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

1 # Определение переменных x, y и граничных условий для них
2 @variable(model, x >= 0)
3 @variable(model, y >= 0)

y

1 # Определение ограничений модели
2 @constraint(model, 6x + 8y >= 100)
3 @constraint(model, 7x + 12y >= 120)

 $7x + 12y \geq 120$ 

1 # Определение целевой функции
2 @objective(model, Min, 12x + 20y)

 $12x + 20y$ 

1 # Вызов функции оптимизации
2 optimize!(model)

1 # Определение причины завершения работы оптимизатора
2 termination_status(model)

OPTIMAL::TerminationStatusCode = 1

```

Рисунок 3.2: Линейное программирование 2

```

1 # Демонстрация прецизионных результирующих значений переменных x и y
2 @show value(x);
3 @show value(y);
4 # Демонстрация результата оптимизации
5 @show objective_value(model);

value(x) = 14.999999999999993
value(y) = 1.25000000000000047
objective_value(model) = 205.0

```

Рисунок 3.3: Линейное программирование 3

Повторим примеры из раздела векторизованные ограничения и целевая функция оптимизации.

```

1 # Подключение пакетов
2 import Pkg
3 Pkg.add("JuMP")
4 Pkg.add("GLPK")
5 using JuMP
6 using GLPK

Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`
Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`

1 # Определение объекта модели с именем vector_model
2 vector_model = Model(GLPK.Optimizer)

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

1 # Определение начальных данных
2 A = [1 1 19 5;
3      3 5 0 8;
4      2 0 6 13]
5 b = [7; 3; 5]
6 c = [1; 3; 5; 2]

4-element Vector{Int64}:
 1
 3
 5
 2

```

Рисунок 3.4: Векторизованные ограничения и целевая функция оптимизации 1

```

1 # Определение вектора переменных
2 @variable(vector_model, x[1:4] >= 0)

4-element Vector{VariableRef}:
 x[1]
 x[2]
 x[3]
 x[4]

```

Рисунок 3.5: Векторизованные ограничения и целевая функция оптимизации 2

```

1 # Определение ограничений модели
2 @constraint(vector_model, A * x .== b)

3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
 x[1] + x[2] + 19 x[3] + 5 x[4] = 7
 3 x[1] + 5 x[2] + 8 x[4] = 3
 2 x[1] + 6 x[3] + 13 x[4] = 5

1 # Определение целевой функции
2 @objective(vector_model, Min, c' * x)

x1 + 3x2 + 5x3 + 2x4

1 optimize!(vector_model)

1 # Определение причины завершения работы оптимизатора
2 termination_status(vector_model)

OPTIMAL::TerminationStatusCode = 1

1 # Демонстрация результата оптимизации
2 @show objective_value(vector_model);

objective_value(vector_model) = 2.3746835443037977

```

Рисунок 3.6: Векторизованные ограничения и целевая функция оптимизации 3

Повторим примеры из раздела оптимизация рациона питания.

```

1 # Подключение пакетов
2 import Pkg
3 Pkg.add("JuMP")
4 Pkg.add("GLPK")
5 using JuMP
6 using GLPK

Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`
Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`

1 # Контейнер для хранения данных об ограничениях
2 # на количество потребляемых калорий, белков, жиров и соли
3 category_data = JuMP.Containers.DenseAxisArray{
4     [1800 2200;
5      91 Inf;
6      0 65;
7      0 1779],
8     ["calories", "protein", "fat", "sodium"],
9     ["min", "max"]}
10 )

2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
  Dimension 2, ["min", "max"]
And data, a 4x2 Matrix{Float64}:
1800.0  2200.0
  91.0    Inf
   0.0   65.0
   0.0 1779.0

1 # Массив данных с наименованиями продуктов
2 foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni",
3 "pizza", "salad", "milk", "ice cream"];

```

Рисунок 3.7: Оптимизация рациона питания 1

```

1 # Массив стоимости продуктов
2 cost = JuMP.Containers.DenseAxisArray(
3     [2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59],
4     foods
5 )

1-dimensional DenseAxisArray{Float64,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni",
  "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
2.49
2.89
1.5
1.89
2.09
1.99
2.49
0.89
1.59

```

Рисунок 3.8: Оптимизация рациона питания 2

```

1 # Массив данных о содержании калорий, белков, жиров и соли в
2 # в продуктах питания
3 food_data = JuMP.Containers.DenseAxisArray(
4     [410 24 26 730;
5      420 32 10 1190;
6      560 20 32 1800;
7      380 4 19 270;
8      320 12 10 930;
9      320 15 12 820;
10     320 31 12 1230;
11     100 8 2.5 125;
12     330 8 10 180],
13     foods,
14     ["calories", "protein", "fat", "sodium"]
15 )

2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni",
  ", "pizza", "salad", "milk", "ice cream"]
  Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9×4 Matrix{Float64}:
410.0  24.0  26.0  730.0
420.0  32.0  10.0  1190.0
560.0  20.0  32.0  1800.0
380.0   4.0  19.0   270.0
320.0  12.0  10.0   930.0
320.0  15.0  12.0   820.0
320.0  31.0  12.0  1230.0
100.0   8.0   2.5   125.0
330.0   8.0  10.0   180.0

1 # Определение объекта модели с именем model
2 model = Model(GLPK.Optimizer)

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

```

Рисунок 3.9: Оптимизация рациона питания 3

```

1 # Определим массив
2 categories = ["calories", "protein", "fat", "sodium"]

4-element Vector{String}:
"calories"
"protein"
"fat"
"sodium"

1 # Определение переменных
2 @variables(model, begin
3     category_data[c, "min"] <= nutrition[c = categories] <=
4     category_data[c, "max"]
5     # Сколько покупать продуктов
6     buy[foods] >= 0
7     end)

(1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{VariableRef}:
nutrition[calories]
nutrition[protein]
nutrition[fat]
nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with
index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni",
  "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{VariableRef}:
buy[hamburger]
buy[chicken]
buy[hot dog]
buy[fries]
buy[macaroni]
buy[pizza]
buy[salad]
buy[milk]
buy[ice cream])

```

Рисунок 3.10: Оптимизация рациона питания 4

```

1 # Определение целевой функции
2 @objective(model, Min, sum(cost[f] * buy[f] for f in foods))

2.49buyhamburger + 2.89buychicken + 1.5buyhotdog + 1.89buyfries + 2.09buymacaroni + 1.99buypizza
+ 0.89buymilk + 1.59buyicecream

1 # Определение ограничений модели
2 @constraint(model, [c in categories],
3 sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])

1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape},1,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
 -nutrition[calories] + 410 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] + 320 buy[pizza] + 320 buy[salad] + 100 buy[milk] + 330 buy[ice cream] = 0
 -nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza] + 31 buy[salad] + 8 buy[milk] + 8 buy[ice cream] = 0
 -nutrition[fat] + 26 buy[hamburger] + 10 buy[chicken] + 32 buy[hot dog] + 19 buy[fries] + 10 buy[macaroni] + 12 buy[pizza] + 12 buy[salad] + 2.5 buy[milk] + 10 buy[ice cream] = 0
 -nutrition[sodium] + 730 buy[hamburger] + 1190 buy[chicken] + 1800 buy[hot dog] + 270 buy[fries] + 930 buy[macaroni] + 820 buy[pizza] + 1230 buy[salad] + 125 buy[milk] + 180 buy[ice cream] = 0

1 # Вызов функции оптимизации
2 JuMP.optimize!(model)
3 term_status = JuMP.termination_status(model)

OPTIMAL::TerminationStatusCode = 1

```

Рисунок 3.11: Оптимизация рациона питания 5

```

1 hcat(buy.data, JuMP.value.(buy.data))

9×2 Matrix{AffExpr}:
 buy[hamburger]  0.6045138888888888
 buy[chicken]    0
 buy[hot dog]    0
 buy[fries]      0
 buy[macaroni]   0
 buy[pizza]      0
 buy[salad]      0
 buy[milk]       6.970138888888935
 buy[ice cream]  2.591319444444441

```

Рисунок 3.12: Оптимизация рациона питания 6

Повторим примеры из раздела путешествие по миру.

```

1 # Подключение пакетов
2 import Pkg
3 Pkg.add("DelimitedFiles")
4 Pkg.add("CSV")
5 using DelimitedFiles
6 using CSV

Resolving package versions...
Updating `~/julia/environments/v1.11/Project.toml`
[8bb1440f] + DelimitedFiles v1.9.1
No Changes to `~/julia/environments/v1.11/Manifest.toml`
Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`

1 # Считывание данных
2 passportdata = readdlm(joinpath("passport-index-dataset",
3 "passport-index-matrix.csv"), ',')

200×200 Matrix{Any}:
"Passport"      "Albania"      ...  "Afghanistan"
"Afghanistan"   "visa required" -1
"Albania"       -1             "visa required"
"Algeria"       "visa required" "visa required"
"Andorra"       90            "visa required"
"Angola"        "visa required" ...  "visa required"
"Antigua and Barbuda" 90            "visa required"
"Argentina"     90            "visa required"
"Armenia"       90            "visa required"
"Australia"     90            "visa required"
"Austria"       90            ...  "visa required"
"Azerbaijan"    90            "visa required"
"Bahamas"       90            "visa required"
⋮
"United Arab Emirates" 90            "visa required"
"United Kingdom"      90            "visa required"
"United States"       90            ...  "visa required"
"Uruguay"             90            "visa required"
"Uzbekistan"          "visa required" "visa required"
"Vanuatu"             "visa required" "visa required"
"Vatican"             90            "visa required"
"Venezuela"           90            ...  "visa required"

```

Рисунок 3.13: Путешествие по миру 1


```

1 # Подключение пакетов
2 import Pkg
3 Pkg.add("JuMP")
4 Pkg.add("GLPK")
5 using JuMP
6 using GLPK

Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`
Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`

1 # Задаём переменные
2 cntr = passportdata[2:end,1]
3 vf = (x -> typeof(x)==Int64 || x == "VF" || x == "VOA" ?
4      1 : 0).(passportdata[2:end, 2:end]);

1 # Определение объекта модели с именем model
2 model = Model{GLPK.Optimizer}

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

1 # Переменные, ограничения и целевая функция
2 @variable(model, pass[1:length(cntr)], Bin)
3 @constraint(model, [j=1:length(cntr)], sum(vf[i, j]*pass[i] for i
4      in 1:length(cntr)) >= 1)
5 @objective(model, Min, sum(pass))

pass1 + pass2 + pass3 + pass4 + pass5 + pass6 + pass7 + pass8 + pass9 + pass10 + pass1
+ pass14 + pass15 + pass16 + pass17 + pass18 + pass19 + pass20 + pass21 + pass22 + pass
+ pass26 + pass27 + pass28 + pass29 + pass30 + [... 139 terms omitted ...] + pass170 + pa
+ pass173 + pass174 + pass175 + pass176 + pass177 + pass178 + pass179 + pass180 + pass181
+ pass184 + pass185 + pass186 + pass187 + pass188 + pass189 + pass190 + pass191 + pass192
+ pass195 + pass196 + pass197 + pass198 + pass199

```

Рисунок 3.14: Путешествие по миру 2

```

1 # Вызов функции оптимизации
2 JuMP.optimize!(model)
3 termination_status(model)

OPTIMAL::TerminationStatusCode = 1

1 # Просмотр результата
2 print(JuMP.objective_value(model), " passports: ",
3      join(cntr[findall(JuMP.value.(pass) .== 1)], ", "))

63.0 passports: Afghanistan, Andorra, Argentina, Australia, Azerbaijan,
Bahrain, Brunei, Cambodia, Cameroon, Canada, Chile, Colombia, Comoros,
DR Congo, Djibouti, Equatorial Guinea, Eritrea, Fiji, Gabon, Georgia, G
uinea, Guinea-Bissau, Hong Kong, Hungary, Indonesia, Iraq, Ireland, Isr
ael, Jamaica, Japan, Kuwait, Laos, Liberia, Libya, Macao, Madagascar, M
alaysia, Maldives, Marshall Islands, Mauritania, Mauritius, Mongolia, M
ozambique, Nauru, Nepal, New Zealand, North Korea, Palestine, Papua New
Guinea, Qatar, Saudi Arabia, Solomon Islands, Somalia, South Sudan, Sri
Lanka, Syria, Taiwan, Timor-Leste, Togo, Turkmenistan, United States, U
ruguay, Vietnam

```

Рисунок 3.15: Путешествие по миру 3

Повторим примеры из раздела портфельные инвестиции.

```
1 # Подключение необходимых пакетов
2 import Pkg
3 Pkg.add("DataFrames")
4 Pkg.add("XLSX")
5 Pkg.add("Plots")
6 Pkg.add("PyPlot")
7 Pkg.add("Convex")
8 Pkg.add("SCS")
9 Pkg.add("Statistics")
10 using DataFrames
11 using XLSX
12 using Plots
13 pyplot()
14 using Convex
15 using SCS
16 using Statistics

Resolving package versions...
No Changes to `~/.julia/environments/v1.11/Project.toml`
No Changes to `~/.julia/environments/v1.11/Manifest.toml`
```

Рисунок 3.16: Портфельные инвестиции 1

```
1 # Считываем данные и размещаем их во фрейм
2 T = DataFrame(XLSX.readtable("data/stock_prices.xlsx", "Sheet2"))
```

13×3 DataFrame

Row	MSFT	FB	AAPL
	Any	Any	Any
1	101.93	137.95	148.26
2	102.8	143.8	152.29
3	107.71	150.04	156.82
4	107.17	149.01	157.76
5	102.78	165.71	166.52
6	105.67	167.33	170.41
7	108.22	162.5	170.42
8	110.97	161.89	172.97
9	112.53	162.28	174.97
10	110.51	169.6	172.91
11	115.91	165.98	186.12
12	117.05	164.34	191.05
13	117.94	166.69	189.95

Рисунок 3.17: Портфельные инвестиции 2

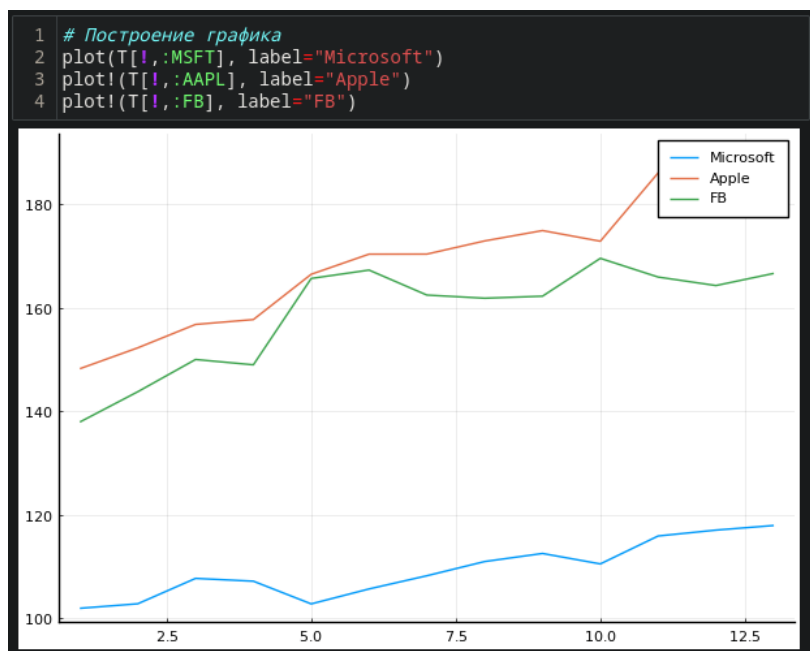


Рисунок 3.18: Портфельные инвестиции 3

```

1 # Данные о ценах на акции размещаем в матрице
2 prices_matrix = Matrix(T)

13×3 Matrix{Any}:
101.93  137.95  148.26
102.8   143.8   152.29
107.71  150.04  156.82
107.17  149.01  157.76
102.78  165.71  166.52
105.67  167.33  170.41
108.22  162.5   170.42
110.97  161.89  172.97
112.53  162.28  174.97
110.51  169.6   172.91
115.91  165.98  186.12
117.05  164.34  191.05
117.94  166.69  189.95

1 # Вычисление матрицы доходностей за период времени
2 M1 = prices_matrix[1:end-1,:]
3 M2 = prices_matrix[2:end,:]
4 # Матрица доходностей
5 R = (M2.-M1)./M1

12×3 Matrix{Float64}:
 0.00853527  0.0424067  0.027182
 0.0477626   0.0433936  0.0297459
-0.00501346 -0.00686484  0.00599413
-0.040963    0.112073   0.0555274
 0.0281183   0.00977611  0.0233606
 0.0241317  -0.0288651    5.8682e-5
 0.0254112  -0.00375385  0.014963
 0.0140579   0.00240904  0.0115627
-0.0179508   0.0451072  -0.0117734
 0.0488644  -0.0213443   0.0763981
 0.00983522 -0.00988071   0.0264883
 0.00760359  0.0142996   -0.00575766

```

Рисунок 3.19: Портфельные инвестиции 4

```
1 # Матрица рисков
2 risk_matrix = cov(R)
3 # Проверка положительной определённости матрицы рисков
4 isposdef(risk_matrix)

true

1 # Доход от каждой из компаний
2 r = mean(R, dims=1)[: ]

3-element Vector{Float64}:
 0.012532748705136576
 0.016563036855293173
 0.021145804655032917

1 # Вектор инвестиций
2 x = Variable(length(r))

Variable
size: (3, 1)
sign: real
vexity: affine
id: 154...682

1 # Объект модели
2 problem = minimize(Convex.quadform(x, risk_matrix),
3 [sum(x) == 1; r'*x >= 0.02; x .>= 0])

Problem statistics
problem is DCP      : true
number of variables : 1 (3 scalar elements)
number of constraints : 5 (5 scalar elements)
number of coefficients : 19
number of atoms      : 14
```

Рисунок 3.20: Портфельные инвестиции 5

```

1 # Находим решение
2 solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 10.45 seconds, 801.209 MiB of
memory allocated

-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 5, constraints m: 12
cones:   z: primal zero / dual free vars: 1
         l: linear vars: 4
         q: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys: sparse-direct-amd-qdldl
          nnz(A): 22, nnz(P): 0

-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
  0 | 1.45e+01 | 1.00e+00 | 2.01e+01 | -9.96e+00 | 1.00e-01 | 3.39e-03
 75 | 1.70e-04 | 3.73e-05 | 5.25e-06 | 4.62e-04 | 1.00e-01 | 3.46e-03
-----
status: solved
timings: total: 3.46e-03s = setup: 2.84e-03s + solve: 6.23e-04s
        lin-sys: 3.09e-05s, cones: 1.63e-05s, accel: 5.19e-06s
-----
objective = 0.000462
-----

Problem statistics
problem is DCP      : true
number of variables : 1 (3 scalar elements)
number of constraints : 5 (5 scalar elements)
number of coefficients : 19
number of atoms      : 14

```

Рисунок 3.21: Портфельные инвестиции 6

```

1 x
Variable
size: (3, 1)
sign: real
vexity: affine
id: 154...682
value: [0.07740709795031134, 0.11606771003983696, 0.8065276266877964]

1 sum(x.value)
1.0000024346779448

1 r' * x.value
1×1 adjoint(::Vector{Float64}) with eltype Float64:
0.0199472331085319

1 x.value .* 1000
3×1 Matrix{Float64}:
 77.40709795031134
116.06771003983695
806.5276266877964

```

Рисунок 3.22: Портфельные инвестиции 7

Повторим примеры из раздела восстановление изображения.

```
1 # Подключение необходимых пакетов
2 import Pkg
3 Pkg.add("ImageMagick")
4 Pkg.add("Images")
5 Pkg.add("Convex")
6 Pkg.add("SCS")
7 using Images
8 using Convex
9 using SCS

Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`
Resolving package versions...
Installed ImageSegmentation — v1.10.0
Installed HistogramThresholding — v0.3.1
Installed TiledIteration — v0.5.0
Installed FFTW — v1.10.0
Installed ScopedValues — v1.5.0
Installed RegionTrees — v0.3.2
Installed Images — v0.26.2
Installed ChunkCodecLibZstd — v1.0.0
Installed SLEEFirates — v0.6.43
Installed RealDot — v0.1.0
Installed CustomUnitRanges — v1.0.2
Installed MetaGraphs — v0.8.1
Installed Polynomials — v4.1.0
Installed JLD2 — v0.6.3
Installed ComputationalResources — v0.3.2
Installed ... — v0.4.5
```

Рисунок 3.23: Восстановление изображения 1

```
1 # Считывание исходного изображения
2 Kref = load("data/khiam-small.jpg")
```



```
1 # Испортим некоторые пиксели
2 K = copy(Kref)
3 p = prod(size(K))
4 missingids = rand(1:p, 400)
5 K[missingids] .= RGBX{N0f8}(0.0, 0.0, 0.0)
6 K
7 Gray.(K)
```



Рисунок 3.24: Восстановление изображения 2


```

1 # Матрица цветов
2 Y = Float64.(Gray.(K));

1 # Сформируем новую матрицу
2 correctids = findall(Y[:,].!=0)
3 X = Convex.Variable(size(Y))
4 problem = minimize(nuclearnorm(X))
5 problem.constraints += X[correctids]==Y[correctids]

[ Warning: Concatenating collections of constraints together with `+` or
`+=` to produce a new list of constraints is deprecated. Instead, use
`vcat` to concatenate collections of constraints.
└ @ Convex ~/.julia/packages/Convex/IPPOR/src/deprecations.jl:129

1-element Vector{Constraint}:
 == constraint (affine)
└ + (affine; real)
  └ index (affine; real)
    └ 283×283 real variable (id: 743...730)
      └ 79689×1 Matrix{Float64}

1 # Находим решение
2 solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 3.51 seconds, 276.839 MiB of
memory allocated

-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem:  variables n: 80090, constraints m: 159779
cones:    z: primal zero / dual free vars: 79689
          nuc: nuclear vars: 80090, nucsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled

```

Рисунок 3.25: Восстановление изображения 3

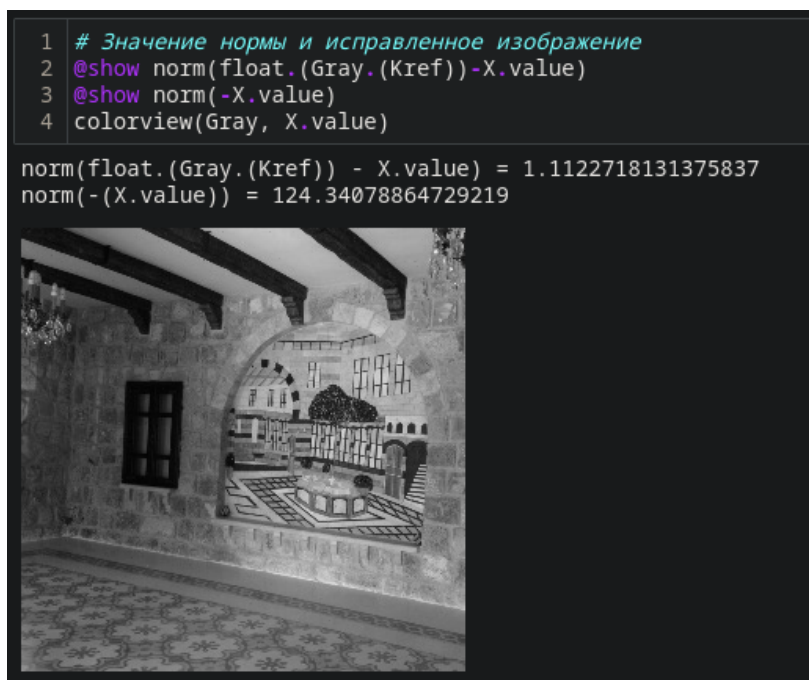


Рисунок 3.26: Восстановление изображения 4

3.2 Задания для самостоятельного выполнения

№1. Линейное программирование.

```

1 # №1
2 # Определение объекта модели с именем model
3 model = Model(GLPK.Optimizer)

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

1 # Определение переменных x, y и граничных условий для них
2 @variable(model, 0 <= x1 <= 10)
3 @variable(model, x2 >= 0)
4 @variable(model, x3 >= 0)

x3

1 # Определение ограничений модели
2 @constraint(model, -x1 + x2 + 3x3 <= -5)
3 @constraint(model, x1 + 3x2 - 7x3 <= 10)

 $x1 + 3x2 - 7x3 \leq 10$ 

1 # Определение целевой функции
2 @objective(model, Max, x1 + 2x2 + 5x3)

 $x1 + 2x2 + 5x3$ 

1 # Вызов функции оптимизации
2 optimize!(model)
3 # Определение причины завершения работы оптимизатора
4 termination_status(model)

OPTIMAL::TerminationStatusCode = 1

```

Рисунок 3.27: Первый номер 1

```

1 # Демонстрация первичных результирующих значений переменных x и y
2 @show value(x1);
3 @show value(x2);
4 @show value(x3);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375

```

Рисунок 3.28: Первый номер 2

№2. Линейное программирование. Использование массивов.

```

1 # №2
2 # Определение объекта модели с именем model
3 vector_model = Model(GLPK.Optimizer)

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

1 A = [-1 1 3; 1 3 -7]
2 b = [-5; 10]
3 c = [1; 2; 5]

3-element Vector{Int64}:
 1
 2
 5

1 @variable(vector_model, x[1:3] >= 0)

3-element Vector{VariableRef}:
 x[1]
 x[2]
 x[3]

1 # Определение ограничений модели
2 @constraint(vector_model, A*x .<= b)
3 @constraint(vector_model, x[1] <= 10)

 $x_1 \leq 10$ 

1 # Определение целевой функции
2 @objective(vector_model, Max, c' * x)

 $x_1 + 2x_2 + 5x_3$ 

```

Рисунок 3.29: Второй номер 1

```

1 # Вызов функции оптимизации
2 optimize!(vector_model)
3 # Определение причины завершения работы оптимизатора
4 termination_status(vector_model)

OPTIMAL::TerminationStatusCode = 1

1 # Демонстрация первичных результирующих значений переменных x и y
2 @show value(x1);
3 @show value(x2);
4 @show value(x3);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375

```

Рисунок 3.30: Второй номер 2

№3. Выпуклое программирование.

```

1  # №3
2  using Random
3  m = 10
4  n = 5
5
6  Random.seed!(42)
7  A = rand(m, n)
8  b = rand(m)
9
10 x = Variable(n)
11
12 objective = sumsquares(A*x-b)
13 constraints = [x >= 0]
14
15 problem = minimize(objective, constraints)

```

Problem statistics

problem is DCP	: true
number of variables	: 1 (5 scalar elements)
number of constraints	: 1 (5 scalar elements)
number of coefficients	: 67
number of atoms	: 6

Solution summary

termination status	: OPTIMIZE_NOT_CALLED
primal status	: NO_SOLUTION
dual status	: NO_SOLUTION

Expression graph

```

minimize
└─ qol (convex; positive)
   └─ + (affine; real)
      └─ * (affine; real)
         └─ ...
            └─ ...
               └─ 10×1 Matrix{Float64}
                  └─ [1;;]
subject to
└─ ≥ constraint (affine)
   └─ + (affine; real)

```

Рисунок 3.31: Третий номер 1

```

1 solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 2.25 seconds, 175.393 MiB of
memory allocated

-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 6, constraints m: 17
cones:    l: linear vars: 5
          q: soc vars: 12, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 57, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
  0 | 1.76e+01 | 1.00e+00 | 2.45e+01 | -1.21e+01 | 1.00e-01 | 1.03e-02
125 | 1.17e-06 | 5.96e-07 | 2.76e-06 | 5.98e-01 | 1.42e+00 | 3.36e-02
-----
status: solved
timings: total: 3.36e-02s = setup: 1.01e-04s + solve: 3.35e-02s
          lin-sys: 6.69e-05s, cones: 2.18e-05s, accel: 8.81e-06s
-----
objective = 0.597986
-----

Problem statistics
problem is DCP      : true
number of variables : 1 (5 scalar elements)
number of constraints : 1 (5 scalar elements)
number of coefficients : 67
number of atoms      : 6

Solution summary
termination status : OPTIMAL

```

Рисунок 3.32: Третий номер 2

```

1 println("Статус задачи: ", problem.status)

Статус задачи: OPTIMAL

```

Рисунок 3.33: Третий номер 3

№4. Оптимальная рассадка по залам.

```

1  # №4
2  num_sections = 5
3  num_applications = 1000
4
5  min_capacity = [180, 180, 220, 180, 180]
6  max_capacity = [250, 250, 220, 250, 250]
7
8  Random.seed!(42)
9
10 priorities = [shuffle([1, 2, 3, 10000, 10000]) for _
11               in 1:num_applications]
12 priorities = hcat(priorities...)'
13 priority_weights = sum(priorities, dims=1)

1×5 Matrix{Int64}:
3851232  4091162  4181199  4031198  3851209

1  model = Model(GLPK.Optimizer)

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

1  @variables(model, begin
2      180 <= x1 <= 250
3      180 <= x2 <= 250
4      x3 == 220
5      180 <= x4 <= 250
6      180 <= x5 <= 250
7      end)

(x1, x2, x3, x4, x5)

```

Рисунок 3.34: Четвёртый номер 1

```

1 @constraint(model, x1+x2+x3+x4+x5==1000)
2 @objective(model, Min, x1 * priority_weights[1] +
3                       x2 * priority_weights[2] +
4                       x3 * priority_weights[3] +
5                       x4 * priority_weights[4] +
6                       x5 * priority_weights[5])
3851232x1 + 4091162x2 + 4181199x3 + 4031198x4 + 3851209x5

1 optimize!(model)
2 termination_status(model)

OPTIMAL::TerminationStatusCode = 1

1 println("x1: ", value(x1))
2 println("x2: ", value(x2))
3 println("x3: ", value(x3))
4 println("x4: ", value(x4))
5 println("x5: ", value(x5))
6 println("Оптимальное значение целевой функции: ",
7         objective_value(model))

x1: 180.0
x2: 180.0
x3: 220.0
x4: 180.0
x5: 240.0
Оптимальное значение целевой функции: 3.9994005e9

```

Рисунок 3.35: Четвёртый номер 2

№5. План приготовления кофе


```

1 # №5
2 coffee = ["Raf", "Cappuccino"]
3 products = ["grains", "milk", "sugar"]
4
5 cost = JuMP.Containers.DenseAxisArray(
6     [400, 300],
7     coffee
8 )
9 coffee_data = JuMP.Containers.DenseAxisArray(
10    [40 140 5;
11     30 120 0],
12    coffee,
13    products
14 )
15 store = JuMP.Containers.DenseAxisArray([500, 2000, 40], products)

1-dimensional DenseAxisArray{Int64,1,...} with index sets:
  Dimension 1, ["grains", "milk", "sugar"]
And data, a 3-element Vector{Int64}:
 500
2000
 40

1 model = Model(GLPK.Optimizer)
2 @variables(model, begin
3     buy[coffee] >= 0
4 end)
5 @objective(model, Max, sum(cost[c] * buy[c] for c in coffee))
6 @constraint(model, [p in products],
7     sum(coffee_data[c, p] * buy[c] for c in coffee) <= store[p])
8 @constraint(model, coffee_data["Raf", "sugar"] * buy["Raf"] == 40)

5buyRaf = 40

```

Рисунок 3.36: Пятый номер 1

```

1 JuMP.optimize!(model)
2 term_status = JuMP.termination_status(model)
3 hcat(buy.data, JuMP.value.(buy.data))

2×2 Matrix{AffExpr}:
 buy[Raf]      8
 buy[Cappuccino] 6

```

Рисунок 3.37: Пятый номер 2

4 Выводы

Освоены пакеты Julia для решения задач оптимизации.

Список литературы

- [1] Julia Language Documentation. *Julia 1.5 Documentation*. 2020. URL: <https://docs.julialang.org/en/v1/>.